APRENDIZAJE AUTOMÁTICO · MCDA 2026-2 · LÍNEA SISTEMAS

Conformal Prediction

Cómo ponerle un margen de seguridad con garantía a cualquier modelo

Pedro Villegas Michel

Profesor: Andrés Vásquez Restrepo · Universidad EAFIT

Octubre 2026

Repositorio: github.com/PedroV27/Conformal-Predictions

HOJA DE RUTA · 20 MINUTOS

Agenda

- **El problema:** un número no basta
- **La idea,** con una analogía
- **Por qué funciona** y los 4 pasos
- **La función central** en Python
- **Tres métodos,** paso a paso y comparados
- **Ejemplo:** el caso de fraude del curso con XGBoost
- **Riesgos** y cómo usarlo con MAPIE
- **Aplicaciones** reales y en mi trabajo

EL PROBLEMA

Un modelo nos da un número… pero ¿cuánto confiar en él?

Lo que entrega hoy el modelo

"El gasto de marzo será **$1.200 M**"
"Esta transacción es fraude con **p = 0,83**"

Lo que necesita quien decide

"El gasto estará entre **$1.050 M y $1.380 M**, y acierto 9 de cada 10 veces"
"Con esta regla detecto **al menos el 95 % de los fraudes**"

- Un número solo **no dice qué tan lejos** puede estar la realidad.
- Las probabilidades de XGBoost sirven para **ordenar** los casos, pero no son confiables como porcentaje (lo vimos en el notebook 02).
- Necesitamos un **margen de seguridad** que se pueda comprobar.

LA IDEA, CON UNA ANALOGÍA

El empleado nuevo que estima facturas

1. Le muestro **100 facturas cuyo valor real ya conozco** y que él nunca ha visto.
2. Anoto **cuánto se equivocó** en cada una.
3. Ordeno los errores de menor a mayor y miro el **error número 90**: digamos, $80 mil.
4. Desde hoy, si dice "vale $1 millón", yo digo: **"entre $920 mil y $1.080.000"**, y acierto el 90 % de las veces.

En la analogía	Nombre técnico
El empleado	El **modelo** (XGBoost)
Las 100 facturas	Set de **calibración**
Cuánto se equivocó	El **score** (error)
El error número 90	**$\hat{q}$**, el umbral
El 90 %	**$1 - \alpha$** (α = error que acepto)

No necesito saber **cómo piensa** el empleado. Solo medí sus errores en casos que no había visto.

LA PROMESA

Qué garantiza conformal prediction

$$P(\text{valor real} \in \text{rango que entrego}) \geq 1 - \alpha$$

Sin supuestos de distribución

No importa si los datos son normales, sesgados o con valores extremos.

Para cualquier modelo

Regresión logística, XGBoost o una red neuronal: se usa como **caja negra**.

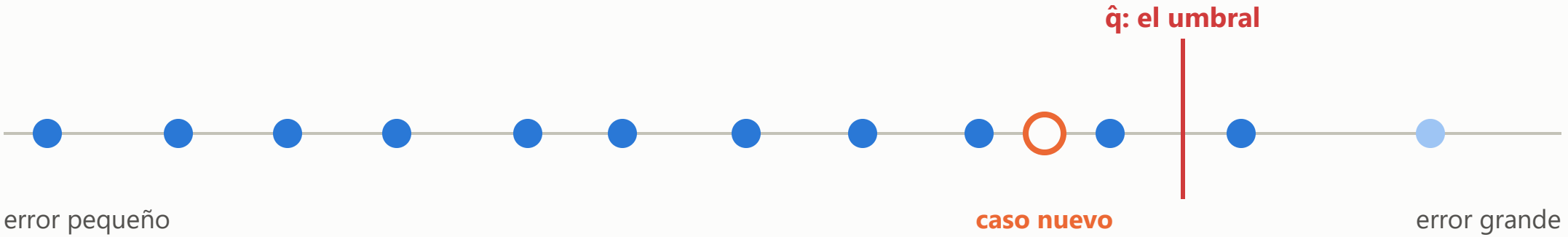
Con pocos o muchos datos

La garantía vale con los datos que tengo, no solo "con infinitos datos".

Un solo requisito: que los datos nuevos **se parezcan** a los de calibración (técnicamente, que sean **intercambiables**). Vovk, Gammerman & Shafer (2005).

POR QUÉ FUNCIONA

Un argumento de ranking



- Pongo en fila los errores de calibración, de menor a mayor.
- El caso nuevo es “uno más del grupo”: su error puede caer en **cualquier posición de la fila con la misma probabilidad**.
- Si pongo el umbral en la posición 90 de 100, solo hay un 10 % de probabilidad de que quede por encima. **Ese es todo el teorema.**

EL ALGORITMO

Siempre los mismos cuatro pasos

1. **Separar** los datos en entrenamiento, **calibración** y prueba.
2. **Entrenar** el modelo solo con los datos de entrenamiento.
3. **Medir los errores** (scores) del modelo en calibración.
4. **Tomar el umbral $\hat{q}$** y usarlo para armar el rango de cada caso nuevo.

$$\hat{q} = \text{el error en la posición } \lceil (n+1)(1-\alpha) \rceil$$

¿Por qué n+1? Porque el caso nuevo también entra en la fila: con 9 errores de calibración hay 10 posiciones posibles.

Resultado garantizado: la cobertura queda entre $1 - \alpha$ y $1 - \alpha + 1/(n+1)$.

Es barato: **un solo entrenamiento** y ordenar n números. Se agrega a cualquier modelo que ya tengamos.

EN CÓDIGO · LA PIEZA CENTRAL

Todo el método cabe en una función

```python
import numpy as np

def conformal_quantile(scores, alpha):
    n = len(scores)                          # cuántos errores tengo
    k = int(np.ceil((n + 1) * (1 - alpha)))  # qué posición del ranking tomo
    if k > n:                                # muy pocos datos:
        return np.inf                        #   no puedo prometer nada
    return np.sort(scores)[k - 1]            # ordeno y devuelvo el k-ésimo
```

```python
errores = np.array([0.3, 1.2, 0.7, 2.5, 0.1, 0.9, 1.8, 0.4, 3.1])
conformal_quantile(errores, alpha=0.2)   # 9 errores, quiero 80 %
# k = (9 + 1) x 0.8 = 8  ->  el 8.º más pequeño  ->  2.5
```

- Recibe los **errores de calibración** y el error que acepto (α).
- **Ordena** los errores y devuelve el de la posición k: ese es **q̂**.
- No mira el modelo ni supone nada sobre los datos.
- **Es la misma función para los tres métodos.** Solo cambia qué llamamos “error”.

LOS TRES MÉTODOS

Lo único que cambia es qué llamamos “error”

Método	¿Qué es el “error”?	¿Qué entrega?
1. Split conformal banda fija	La distancia entre el valor real y la predicción	Predicción ± q̂: **el mismo margen para todos**
2. CQR banda adaptativa	Cuánto se sale el valor real de una banda “piso–techo”	Un margen **ancho donde hay duda y angosto donde no**
3. Mondrian umbral por clase	1 − la probabilidad que el modelo le dio a la respuesta correcta, **medido por separado en cada clase**	La garantía se cumple **dentro de cada clase** (por ejemplo, en los fraudes)

Vamos uno por uno: primero el código paso a paso y después la comparación de resultados.

MÉTODO 1 · PASO A PASO

Split conformal: predicción ± un margen fijo

```python
from xgboost import XGBRegressor

# Paso 1. Entreno el modelo (solo con train)
model = XGBRegressor(max_depth=3).fit(X_train, y_train)

# Paso 2. Mido sus errores en calibración
errores = np.abs(y_cal - model.predict(X_cal))

# Paso 3. Saco el umbral q_hat
q_hat = conformal_quantile(errores, alpha=0.10)

# Paso 4. Predicción ± q_hat
pred = model.predict(X_new)
lower, upper = pred - q_hat, pred + q_hat
```

- Son exactamente los **cuatro pasos** de antes.
- El "error" es **cuánto se equivocó** el modelo: |real − predicho|.
- **q̂** es el error que solo se supera el 10 % de las veces.
- El intervalo final es la predicción **más y menos q̂**.

Limitación: el margen es **igual para todos** los casos, sean fáciles o difíciles.

MÉTODO 2 · PASO A PASO

CQR: un margen que se adapta a cada caso

```python
# Paso 1. Dos XGBoost: uno para el "piso" y otro para el "techo"
q = dict(objective="reg:quantileerror", max_depth=3)
piso  = XGBRegressor(quantile_alpha=0.05, **q).fit(X_train, y_train)
techo = XGBRegressor(quantile_alpha=0.95, **q).fit(X_train, y_train)

# Paso 2. Error = cuánto se sale el dato real de la banda
errores = np.maximum(piso.predict(X_cal) - y_cal,
                     y_cal - techo.predict(X_cal))

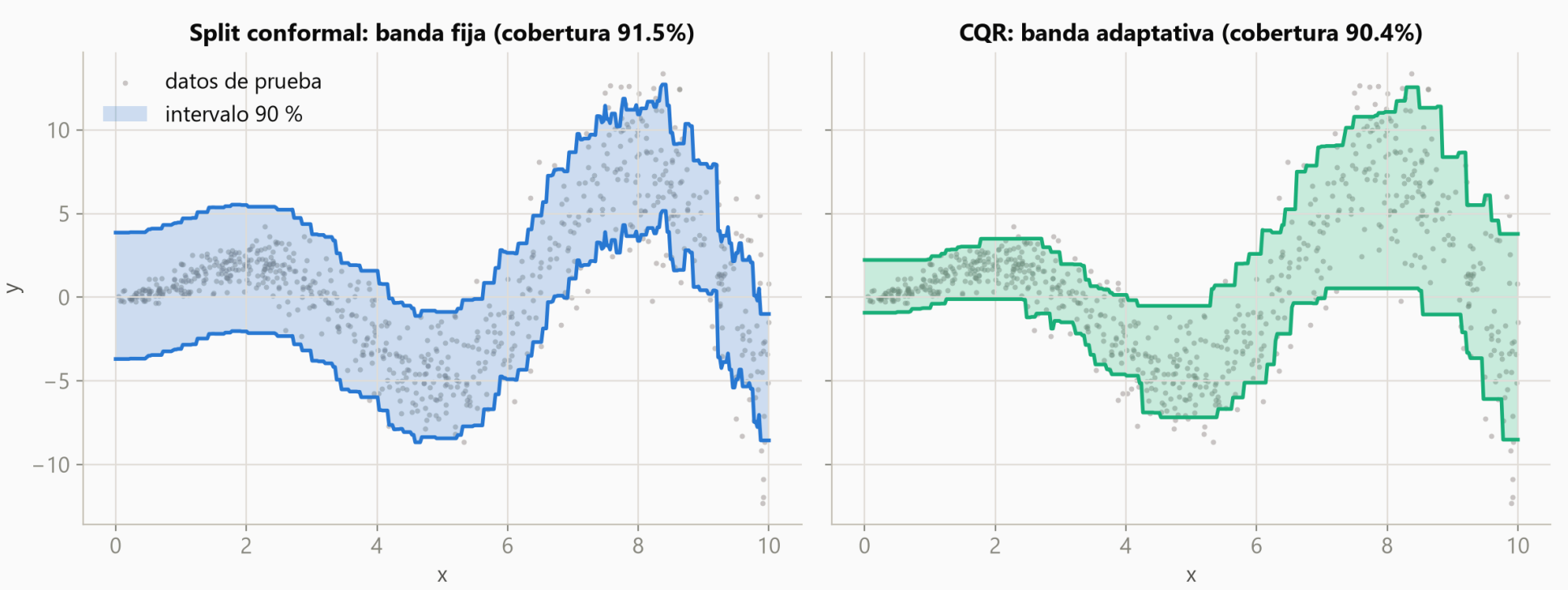
# Paso 3. Saco el umbral q_hat
q_hat = conformal_quantile(errores, alpha=0.10)

# Paso 4. Corrijo la banda con q_hat
lower = piso.predict(X_new) - q_hat
upper = techo.predict(X_new) + q_hat
```

- Entreno **dos XGBoost**: uno predice el “piso” (5 %) y otro el “techo” (95 %).
- Esa banda ya es ancha donde hay duda, pero **no trae garantía**.
- El “error” es **cuánto se sale** el dato real de la banda.
- Con q̂ **corrijo** la banda: la ensancho si cubría de menos o la estrecho si cubría de más.

COMPARACIÓN · MÉTODOS 1 Y 2

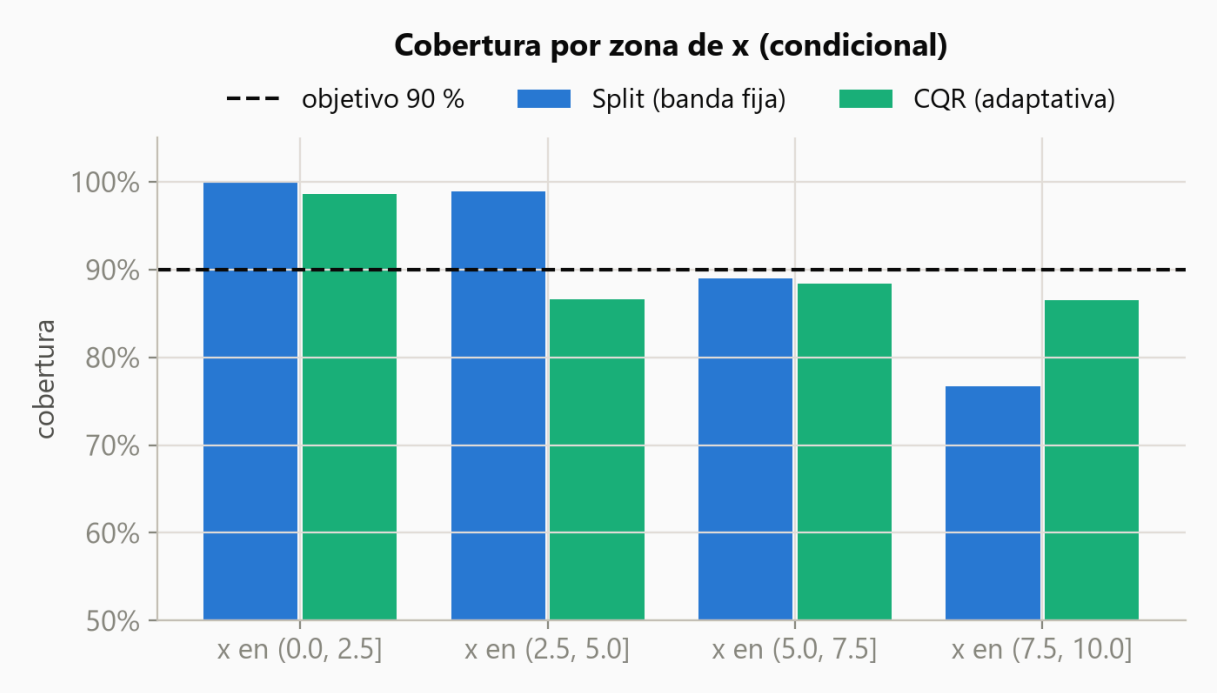
Mismos datos, dos márgenes distintos



Método	¿Cumple el 90 %?	Ancho promedio del intervalo	Lectura
1. Split conformal (banda fija)	Sí: 91,5 %	7,56	Mismo ancho en todas partes
2. CQR (banda adaptativa)	Sí: 90,4 %	**6,61**	**12 % más angosto**: se abre solo donde hace falta

ADVERTENCIA CLAVE

La garantía es un promedio, no se cumple en cada zona



- La banda fija acierta el **100 %** donde hay poco ruido y solo el **77 %** donde hay mucho.
- En promedio da 91 %: **cumple, pero de forma desigual**.
- CQR sube la zona difícil a **87 %**.
- Si el grupo que me importa es una **clase** (como los fraudes), necesito el método 3.

CLASIFICACIÓN

Ahora la respuesta es un conjunto de etiquetas

El modelo ya no dice “es fraude” o “no es fraude”. Dice: **“estas son las respuestas que no puedo descartar”**.

{legítima}
Seguro de que es legítima.
No alerto.

{fraude} o {legítima, fraude}
No puedo descartar fraude.
Alerto para revisión.

{ } vacío
Caso muy raro: ninguna respuesta convence.
Aquí no alerto.

Regla de decisión: alerto si “fraude” está en el conjunto. Así puedo medir con las **métricas del curso**: recall, precisión y matriz de confusión.

MÉTODO 3 · PASO A PASO (1 DE 2)

Primero: medir el error en cada transacción

```
# Paso 1. Entreno el XGBoost del notebook 02 (solo con train)
clf = XGBClassifier(n_estimators=300, max_depth=6,
                    scale_pos_weight=spw,
                    enable_categorical=True).fit(X_train, y_train)

# Paso 2. Mido el error en cada transacción de calibración
P_cal = clf.predict_proba(X_cal)    # 2 columnas: [p(legítima), p(fraude)]

# de cada fila tomo la probabilidad de su clase REAL...
p_real = P_cal[np.arange(len(y_cal)), y_cal]

# ...y el error es lo que le faltó para llegar a 1
s_cal = 1 - p_real
```

El error es lo que le faltó al modelo para darle 100 % a la respuesta correcta. Tres casos de ejemplo:

Era…	El modelo dijo p(fraude) =	Error
Legítima	0,001	**0,001**: acertó
Fraude	0,98	**0,02**: acertó
Fraude	0,10	**0,90**: se equivocó

Error cerca de **0** = el modelo acertó con seguridad.
Error cerca de **1** = el modelo se equivocó.

MÉTODO 3 · PASO A PASO (2 DE 2)

Después: un umbral para cada clase

```
# Paso 3. Saco el umbral q_hat
q_unico   = conformal_quantile(s_cal, 0.05)              # MÉTODO 1: con todos
q_legit   = conformal_quantile(s_cal[y_cal == 0], 0.05)  # MÉTODO 3: solo legítimas
q_fraude  = conformal_quantile(s_cal[y_cal == 1], 0.05)  # MÉTODO 3: solo fraudes

# Paso 4. Una etiqueta entra al conjunto si su error (1 - p) es <= su umbral
P = clf.predict_proba(X_test)
conjunto_m1 = (1 - P) <= q_unico                # mismo umbral para las 2 etiquetas
conjunto_m3 = (1 - P) <= [q_legit, q_fraude]    # un umbral para cada etiqueta

# Decisión: alerto si "fraude" (columna 1) quedó dentro del conjunto
alerta = conjunto_m3[:, 1]
recall_score(y_test, alerta), precision_score(y_test, alerta)
```

Método 1: un solo umbral

$\hat{q}$ = 0,003, calculado con las 60.000. Solo alerta si p(fraude) ≥ **0,997**: casi nunca.

Método 3: Mondrian

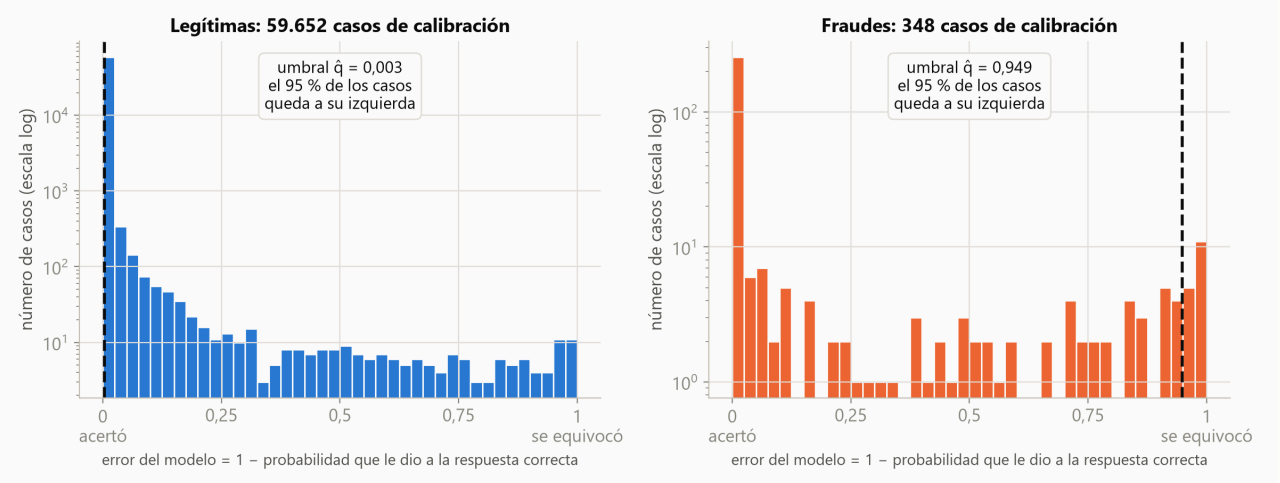
Legítimas: $\hat{q}$ = 0,003 (59.652 casos).
Fraudes: $\hat{q}$ = 0,949 (348 casos).
Alerta si p(fraude) ≥ **0,051**.

Es el mismo código. La única diferencia: Mondrian calcula el umbral de los fraudes **solo con los fraudes**, y por eso es mucho menos exigente para alertar.

EJEMPLO · FRAUDE CON TARJETA (DATASET DEL CURSO)

El montaje del ejercicio

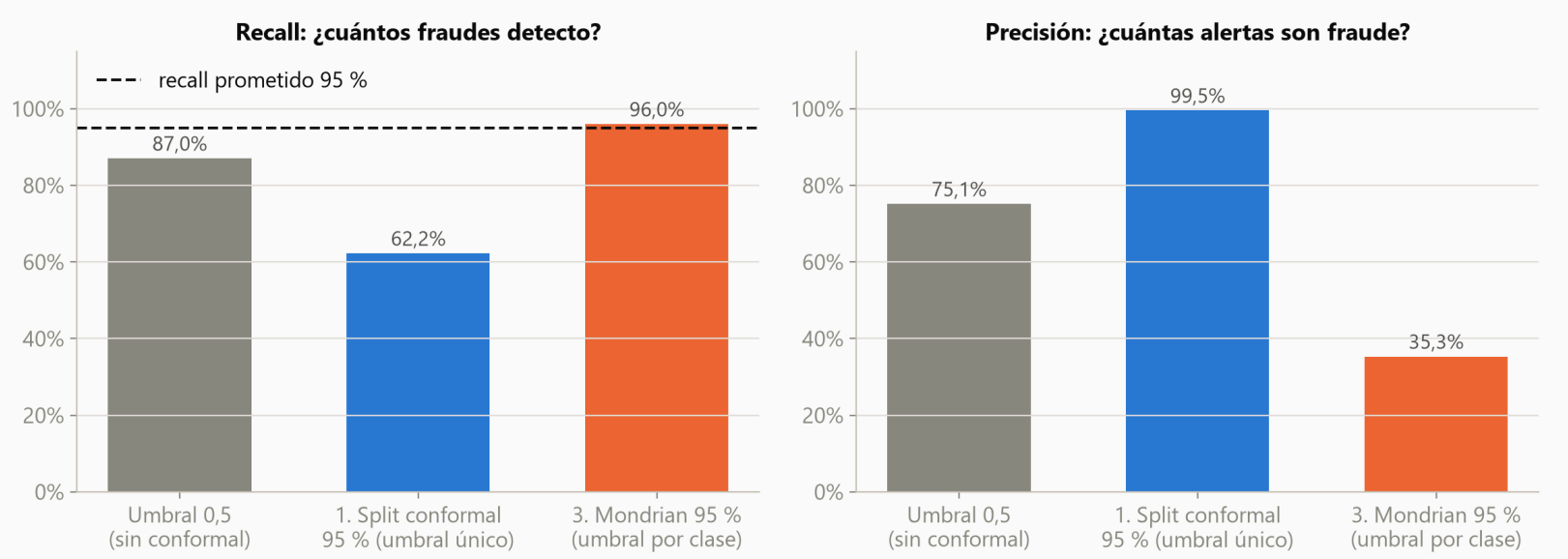
1. **Datos:** 300.000 transacciones, solo 0,58 % son fraude.
2. **Reparto:** 180 mil para entrenar, 60 mil para calibrar y 60 mil para probar.
3. **Modelo:** el XGBoost del notebook 02 (PR-AUC 0,912).
4. **Promesa pedida:** 95 %.



- Cada barra cuenta cuántas transacciones de calibración tuvieron ese error. La línea punteada es el umbral q̂.
- **Legítimas:** casi todas en 0; el modelo las acierta fácil. Umbral muy pequeño.
- **Fraudes:** la mayoría en 0, pero varios cerca de 1: fraudes que el modelo no vio. Umbral grande.

COMPARACIÓN · MÉTODOS 1 Y 3 (DATOS DE PRUEBA)

¿Cuántos fraudes detecta cada regla?



Regla de decisión	Recall	Precisión	Alertas	¿Qué promete?
Umbral 0,5 (sin conformal)	87,0 %	75,1 %	402	Nada
1. Split conformal 95 %	62,2 %	99,5 %	217	95 % en el total (95,3 %), no en los fraudes
3. Mondrian 95 %	**96,0 %**	35,3 %	943	**Detectar ≥ 95 % de los fraudes**

¿POR QUÉ PASA ESTO?

El 99 % legítimo “tapa” a los fraudes

Método 1: un solo umbral

Mezcla todos los errores. Como casi todo es legítimo, el umbral lo deciden las legítimas.

La promesa del 95 % se cumple **en el total**, pero solo se detecta el **62 %** de los fraudes.

Método 3: Mondrian

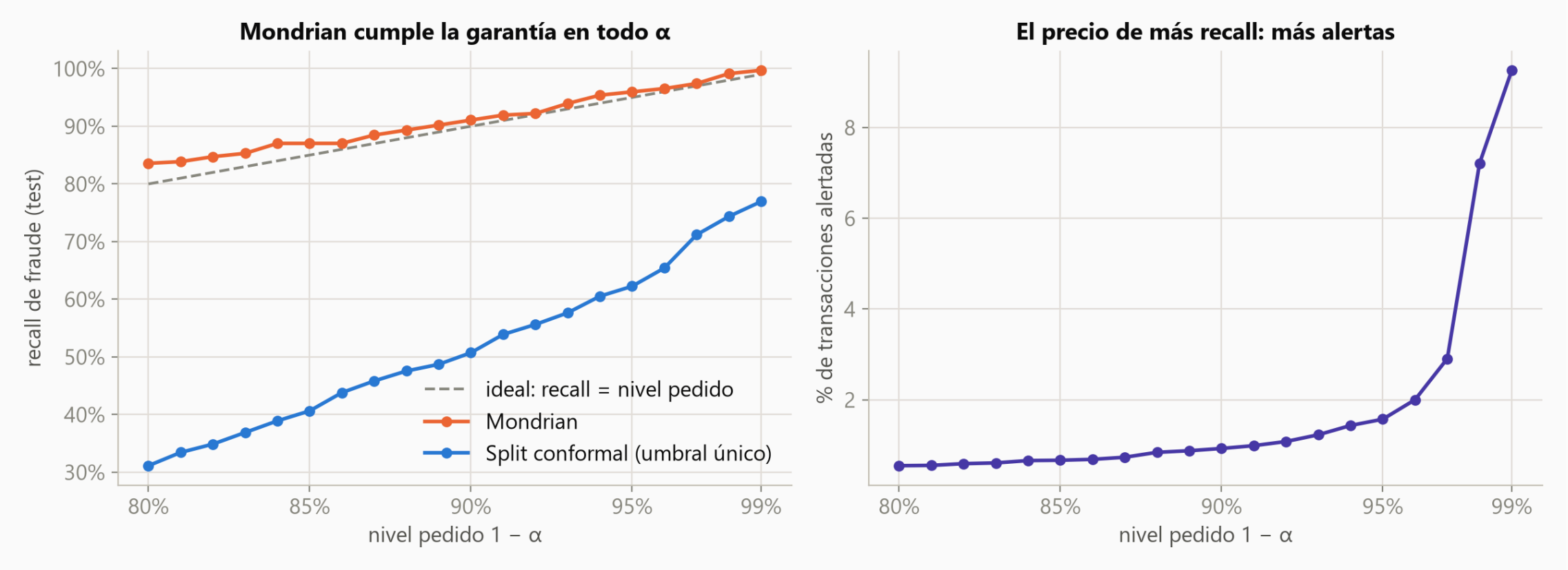
Calcula un umbral **solo con los fraudes** y otro solo con las legítimas.

La promesa se cumple **en cada clase**: detecta el **96 %** de los fraudes. Eso es una **garantía de recall**.

El precio: más alertas (943 frente a 217) y menos precisión. A cambio, **casi ningún fraude se escapa**.

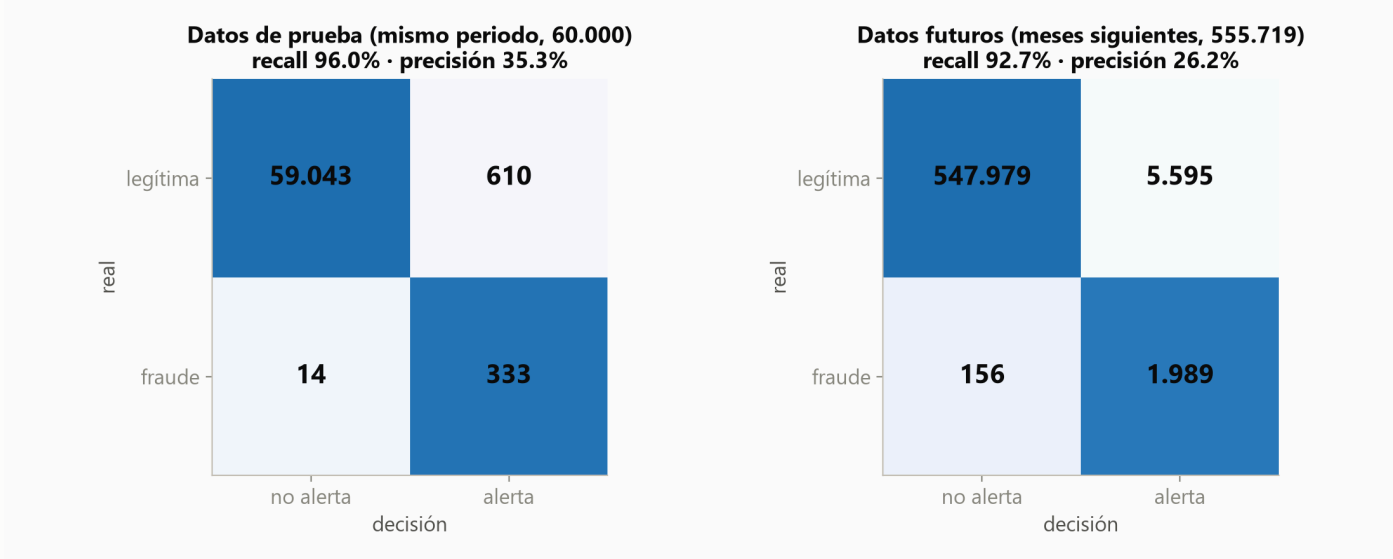
EJEMPLO · EL PRECIO DE PEDIR MÁS

A más recall prometido, más alertas



Mondrian cumple lo que promete en cualquier nivel (sigue la diagonal). Pero prometer 99 % en vez de 95 % sube las alertas de **1,6 %** a **9,3 %** de las transacciones: unas **6 veces más**.

¿La promesa se sostiene en el futuro?



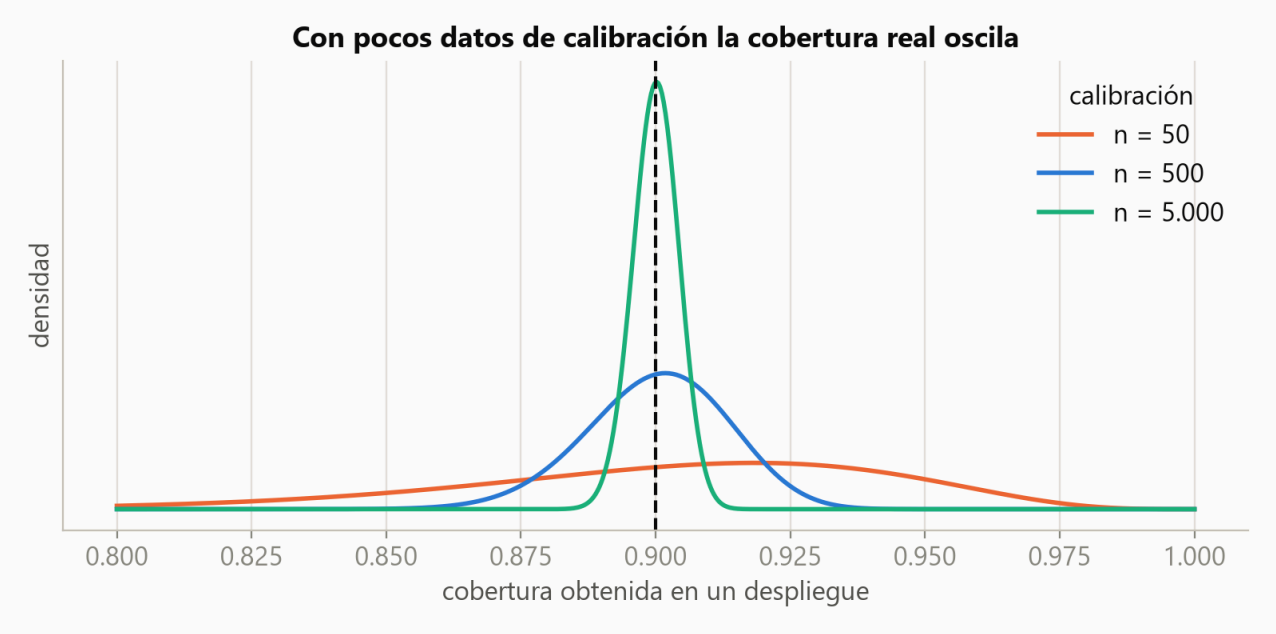
- **Izquierda, datos de prueba:** 60.000 transacciones del mismo periodo, que el modelo no usó ni para entrenar ni para calibrar. Recall 96,0 %: cumple.
- **Derecha, datos futuros:** los meses siguientes (`fraudTest.csv`). Recall **92,7 %**: menos del 95 % prometido.
- ¿Por qué? Los datos cambiaron: el fraude pasó de 0,58 % a 0,39 % y ya **no se parecen** a los de calibración.
- **Lección:** hay que vigilar el recall y **recalibrar** con datos recientes.

RIESGOS DE USO

Cuándo la garantía deja de valer, o engaña

Riesgo	Qué pasa	Qué hacer
Los datos cambian con el tiempo	La promesa se debilita (lo vimos: 92,7 % en vez de 95 %).	Vigilar y recalibrar con datos recientes
La promesa es un promedio	Se cumple en total, pero puede fallar en un grupo.	CQR si la duda cambia; Mondrian si importa una clase
Pocos datos de calibración	El resultado real oscila alrededor de lo prometido.	Idealmente 1.000 o más (por clase, en Mondrian)
Usar calibración para entrenar	El modelo “ya vio” esos datos y la promesa se rompe.	Separar tres partes y usar la calibración una sola vez
Explicarlo mal	“Este intervalo tiene 90 % de probabilidad” es incorrecto.	Decir: “este método acierta el 90 % de las veces”

¿Cuántos datos de calibración necesito?



Si prometo 90 %, ¿qué obtengo en realidad? Depende de cuántos datos usé para calibrar:

Datos de calibración	Lo que realmente obtengo
50	entre 83 % y 96 %
500	entre 88 % y 92 %
5.000	entre 89 % y 91 %

En el ejemplo, Mondrian calibró los fraudes con solo 348 casos: por eso este número importa.

PARA SUS PROYECTOS

Lo mismo con la librería MAPIE

```python
# pip install mapie xgboost
from mapie.regression import SplitConformalRegressor, ConformalizedQuantileRegressor
from mapie.classification import SplitConformalClassifier

# MÉTODO 1 en regresión: banda fija
reg = SplitConformalRegressor(model, confidence_level=0.90, prefit=True)
reg.conformalize(X_cal, y_cal)                      # calibrar
y_pred, intervalos = reg.predict_interval(X_test)   # predecir con intervalo

# MÉTODO 2: CQR, con los XGBoost [piso, techo, mediana]
cqr = ConformalizedQuantileRegressor([piso, techo, mediana],
                                     confidence_level=0.90, prefit=True)
cqr.conformalize(X_cal, y_cal)
y_pred, intervalos = cqr.predict_interval(X_test)

# MÉTODO 1 en clasificación: conjuntos con un umbral único
cls = SplitConformalClassifier(clf, confidence_level=0.95, prefit=True)
cls.conformalize(X_cal, y_cal)
y_pred, conjuntos = cls.predict_set(X_test)
```

Siempre dos pasos

`conformalize` para calibrar y `predict_interval` o `predict_set` para predecir.

Verificado

Con los XGBoost del ejemplo da **los mismos resultados** que el código hecho a mano.

Mondrian

No viene incluido: se hace a mano como en la diapositiva 16.

APLICACIONES REALES

¿Dónde se usa?

Descubrimiento de fármacos

AstraZeneca lo usa para saber en cuáles predicciones sobre moléculas puede confiar (Norinder et al., 2014).

Pronóstico de ventas y energía

Las empresas pronostican la demanda como un rango y no como un solo número. Librerías como MAPIE ya lo incluyen.

Medicina

El modelo entrega una lista corta de diagnósticos posibles, con garantía, y el especialista toma la decisión final.

Riesgo y fraude

Los bancos priorizan qué casos revisar y garantizan que detectan la mayoría de los fraudes, como en nuestro ejemplo.

Modelos de lenguaje

Un chatbot entrega varias respuestas y se garantiza que al menos una es correcta (Quach et al., 2024).

Vehículos autónomos

El vehículo calcula la zona donde puede estar un peatón, con garantía, y así planea una ruta segura.

APLICACIÓN EN MI TRABAJO · GASTOS ADMINISTRATIVOS EN UNA ASEGURADORA

Clasificar nuevas combinaciones contables con garantía

Bajo **NIIF 17**, cada combinación centro de costos–cuenta se clasifica como **atribuible** o **no atribuible** a los contratos de seguro, según la política corporativa.

Un XGBoost aprende la clasificación vigente y **Mondrian** decide qué combinaciones nuevas se asignan solas, con garantía en **cada** categoría.

{Atribuible} o {No atribuible}

Asignación automática y consistente con la política.

{Atribuible, No atribuible}

Caso fronterizo: pasa a revisión del experto, con la sugerencia del modelo.

Y para el presupuesto: con **CQR**, entregar un rango confiable en lugar de un número único.

CIERRE

Tres ideas para llevarse

> 1. Conformal le pone una garantía a cualquier modelo: solo mide sus errores en datos que no vio.

> 2. Banda fija para empezar, CQR cuando la duda cambia y Mondrian cuando una clase importa más.

> 3. La garantía vale mientras los datos no cambien: en producción, vigilar y recalibrar.

BIBLIOGRAFÍA

Referencias

- Angelopoulos, A. N. & Bates, S. (2023). Conformal prediction: A gentle introduction. *Foundations and Trends in ML*, 16(4).
- Chen, T. & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD*.
- Lei, J., G'Sell, M., Rinaldo, A., Tibshirani, R. J. & Wasserman, L. (2018). Distribution-free predictive inference for regression. *JASA*, 113(523).
- Norinder, U. et al. (2014). Introducing conformal prediction in predictive modeling. *J. Chem. Inf. Model.*, 54(6).
- Papadopoulos, H., Proedrou, K., Vovk, V. & Gammerman, A. (2002). Inductive confidence machines for regression. *ECML*.
- Quach, V. et al. (2024). Conformal language modeling. *ICLR*.
- Romano, Y., Patterson, E. & Candès, E. J. (2019). Conformalized quantile regression. *NeurIPS*.
- Vovk, V. (2012). Conditional validity of inductive conformal predictors. *ACML*.
- Vovk, V., Gammerman, A. & Shafer, G. (2005; 2.ª ed. 2022). *Algorithmic Learning in a Random World*. Springer.
- MAPIE: Model Agnostic Prediction Interval Estimator (v1.5). github.com/scikit-learn-contrib/MAPIE

¿Preguntas?

Código reproducible: github.com/PedroV27/Conformal-Predictions

Dataset: Credit Card Transactions Fraud Detection (Kaggle, kartik2112)

Python · NumPy · scikit-learn · XGBoost · MAPIE · matplotlib